

Applied Machine Learning

Introduction to Convolutional Neural Networks

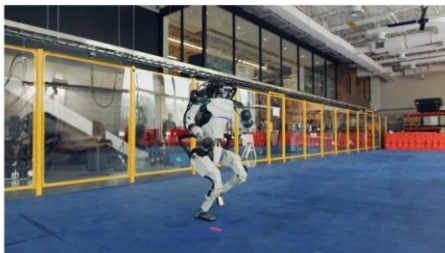


UNIVERSITY OF
GOTHENBURG

CHALMERS

The rise and impact of computer vision

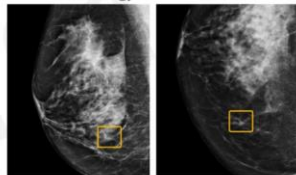
Robotics



Accessibility



Biology & Medicine



Autonomous driving



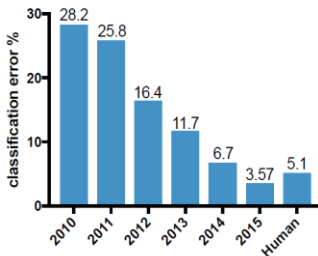
Mobile computing



CNNs: motivation

- ▶ the introduction of **convolutional neural network** (CNN) models has led to dramatic improvements in image processing

ImageNet Challenge: Classification Task



2012: AlexNet. First CNN to win.

- 8 layers, 61 million parameters

2013: ZFNet

- 8 layers, more filters

2014: VGG

- 19 layers

2014: GoogLeNet

- "Inception" modules
- 22 layers, 5 million parameters

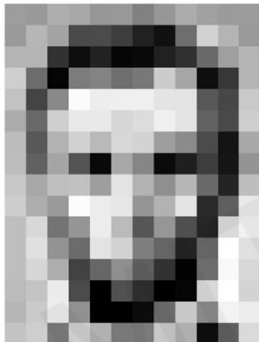
2015: ResNet

- 152 layers

- ▶ they are the **default solution today**

What computers “See”

Images are Numbers

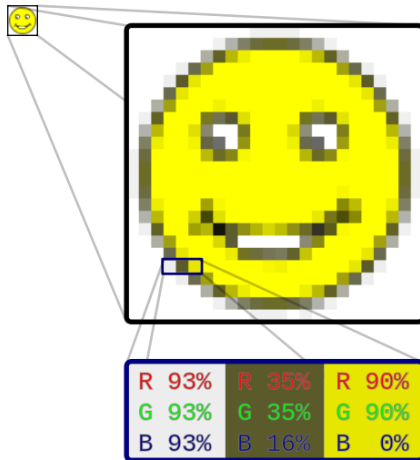


157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	93	17	110	210	180	154
180	180	50	14	34	6	10	93	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	95	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

What the computer sees

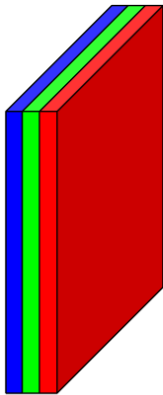
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	85	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	95	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

representing image data



[[source](#)]

representing image data (3)



RGB
Image
 $M \times N \times 3$



Grayscale/Binary
Image
 $M \times N \times 1$

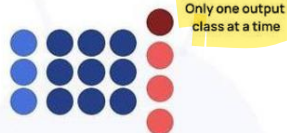
[\[source\]](#)

file formats for image data

- ▶ there are many **file formats** for images:
 - ▶ “raw”/direct formats: BMP, TIFF, ...
 - ▶ compressed lossless formats: PNG, GIF, ...
 - ▶ compressed lossy formats: JPEG, ...
- ▶ also: **domain-specific** image formats (radar, medical, ...)
- ▶ this does not matter to us: we just need the **pixels!**

Tasks in computer vision

Multi-Class



Input Image



167	169	174	168	190	162	120	161	172	161	165	166
185	183	183	74	76	63	33	17	110	210	180	164
180	189	90	14	34	6	10	33	40	106	169	181
206	109	6	124	121	111	120	204	196	15	96	180
194	68	137	251	237	239	239	228	227	67	71	221
173	106	207	239	239	214	230	239	239	98	74	206
180	88	179	209	185	215	211	159	139	75	59	169
189	97	168	84	10	168	134	11	31	62	22	149
199	108	191	193	198	227	178	143	182	166	36	190
208	174	198	292	236	231	149	178	228	43	96	234
190	216	116	149	206	187	86	153	79	39	218	241
190	234	147	108	227	210	127	102	36	101	266	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	236	76	1	81	47	0	6	217	266	211
183	203	237	145	0	0	12	108	206	188	243	236
196	206	123	207	177	121	123	200	176	13	96	218

Pixel Representation

classification

Lincoln

0.8

Washington

0.1

Jefferson

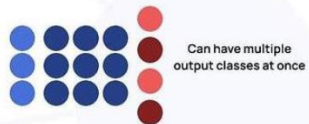
0.05

Obama

0.05

Tasks in computer vision

Multi-Label



how can we **create ML models that work with images?**

can we use the pixels directly as features?



What We See

```
08 02 22 97 38 15 00 40 00 75 04 05 07 78 52 12 50 77 91 08
49 49 99 40 17 51 18 57 60 87 17 40 98 43 69 48 04 56 42 00
81 49 31 73 55 79 14 29 93 71 40 47 53 88 30 03 49 13 36 45
52 70 95 23 04 60 11 42 69 24 68 54 01 32 56 71 37 02 36 91
22 31 16 71 51 47 63 89 41 92 36 54 22 40 40 28 66 33 13 80
24 47 32 60 99 03 45 02 44 75 33 53 78 36 84 20 35 17 12 50
32 98 81 28 64 23 67 10 26 38 40 67 59 54 70 66 18 38 64 70
47 24 20 68 02 62 12 20 95 63 94 39 63 08 40 91 66 49 94 21
24 55 58 05 46 73 99 24 97 17 78 79 96 83 14 88 34 89 43 72
21 16 23 09 75 00 76 44 20 45 35 14 00 41 33 97 34 31 33 95
78 17 53 28 22 75 31 47 15 94 03 80 04 42 16 14 09 53 56 92
16 39 05 42 84 35 31 47 55 58 88 24 00 17 54 24 36 29 85 57
86 56 00 48 35 71 89 07 05 44 44 37 44 60 21 58 51 54 17 58
19 40 81 68 05 94 47 69 28 73 92 13 84 52 17 77 04 89 55 40
04 52 08 93 97 35 99 16 07 97 37 32 16 26 26 79 33 27 98 46
88 34 48 87 57 62 20 72 03 46 33 47 44 55 12 32 43 93 13 49
04 42 16 73 38 25 39 11 24 94 72 10 08 46 29 32 40 42 74 36
20 49 34 41 72 30 23 88 34 62 99 49 82 47 59 85 74 04 36 16
20 73 35 29 78 31 90 01 74 31 49 71 48 86 81 16 23 57 05 34
01 70 54 71 83 51 54 49 16 92 33 48 61 43 52 01 89 19 47 48
```

What Computers See

- ▶ **linear models** can't model interactions between features
- ▶ **decision trees** and **tree ensembles** can handle feature interactions, but would have to be huge!

Manual Feature Extraction

Domain knowledge

Define features

Detect features
to classify

Viewpoint variation



Scale variation



Deformation



Occlusion



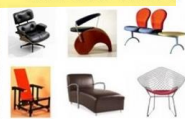
Illumination conditions



Background clutter



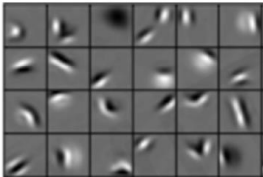
Intra-class variation



Learning Feature Representations

Can we learn a **hierarchy of features** directly from the data instead of hand engineering?

Low level features



Edges, dark spots

Mid level features



Eyes, ears, nose

High level features



Facial structure

reducing the amount of feature engineering

- ▶ neural networks (NNs) are systems that learn to form useful abstractions automatically
- ▶ appealing because of reduction in feature engineering effort

simple idea: just apply a feedforward neural network

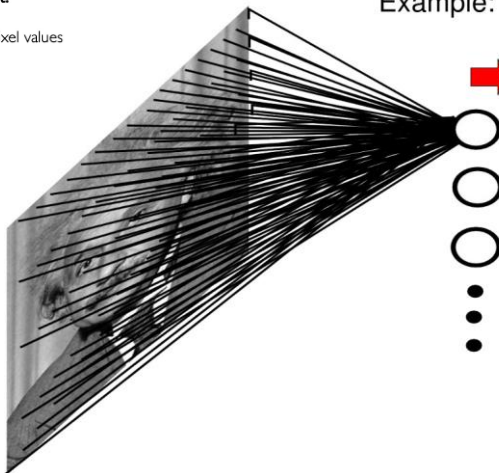
Many, many
parameters

Input:

- 2D image
- Vector of pixel values

Example: 200x200 image
40K hidden units

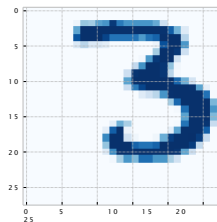
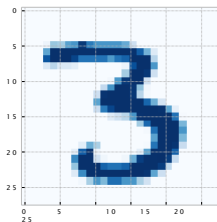
→ ~2B parameters!!!



Idea: Connect neuron in hidden layer to all neurons in input layer [\[source\]](#)

drawbacks of using a FFNN (2)

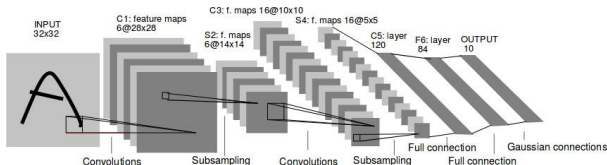
If we look at the pixels directly, we will fail to classify an X as an X if it's shifted, shrunk, rotated, deformed.



- ▶ the model lacks **translational invariance**

convolutional neural networks in a nutshell

- ▶ key idea: **spotting patterns** in images (or other data)
 - ▶ low-level patterns → high-level patterns
 - ▶ these patterns are **learned**
- ▶ like any neural network, it is built as a **stacked architecture**



Applied Machine Learning

Building Blocks of Convolutional Neural Networks

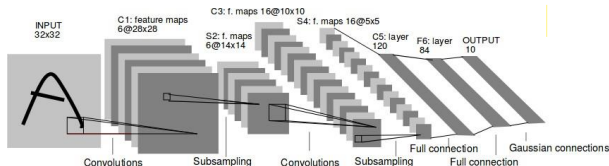


UNIVERSITY OF
GOTHENBURG

CHALMERS

the building blocks of CNNs

- ▶ **convolutional filters**: spotting patterns
- ▶ **pooling**: summarizing regions
- ▶ **fully connected** or **dense** layers: like a regular NN
- ▶ To allow training of deep networks, consider **residual connections** and **batch normalizations**



convolutions (or convolutional filters)

Local receptive field

0	0	0	0	0	0	0	0
0	0	21	0	0	0	0	0
0	85	71	0	0	0	0	0
0	250	231	127	63	3	0	0
0	250	252	250	209	56	0	0
0	250	252	250	250	83	0	0
0	0	0	0	0	0	0	0

Image

$\odot$

0	0	1
0	1	0
1	0	0

Kernel

$\rightarrow$

0	106		

Feature map

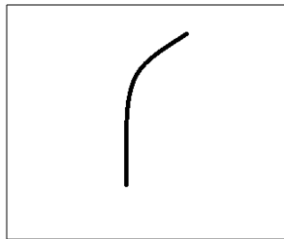
The red cell is the result of a dot product of the red receptive field in the left matrix with the kernel.

[source]

example of a convolution (1)

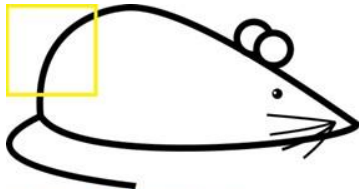
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

example of a convolution (2)



Visualization of the filter on the image

example of a convolution (3)

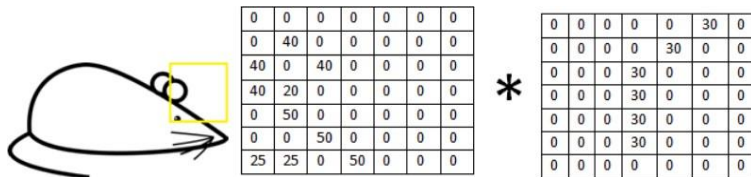


0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

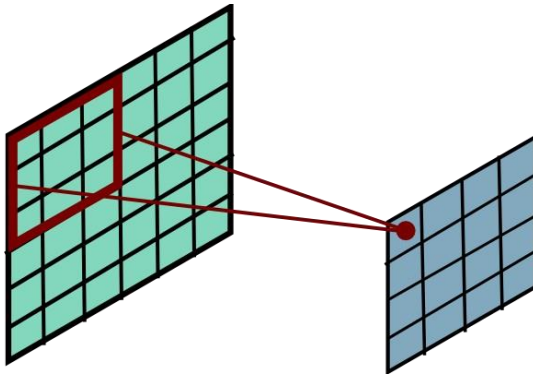
*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

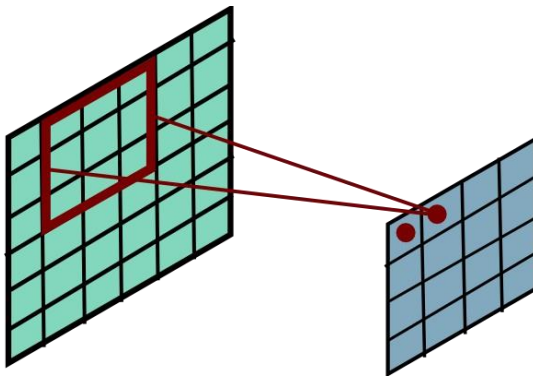
example of a convolution (4)



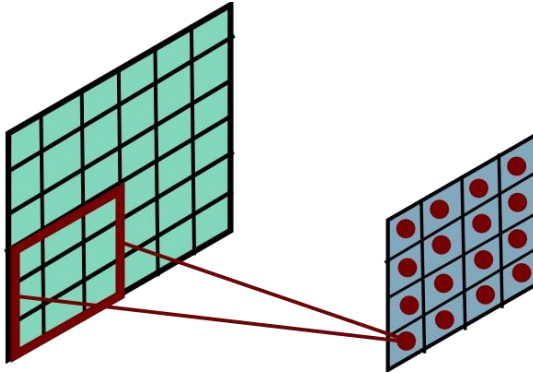
Convolutional Layer



Convolutional Layer



Convolutional Layer



Mathieu et al. "Fast training of CNNs through FFTs" ICLR 2014

example: detecting horizontal and vertical edges

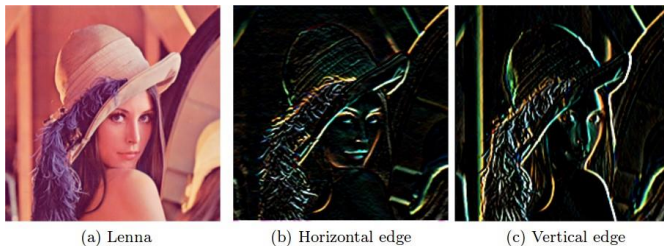


Figure 4: The Lenna image and the effect of different convolution kernels.

[\[source\]](#)

Importance of convolutional layers

X or X?

?

=

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	-1	-1	-1
-1	-1	1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	-1	-1	1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

-1	-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	1	-1	-1
-1	1	-1	-1	-1	1	-1	-1	-1
-1	-1	1	1	-1	1	-1	-1	-1
-1	-1	-1	-1	1	-1	-1	-1	-1
-1	-1	-1	1	-1	1	1	-1	-1
-1	-1	-1	1	-1	-1	-1	1	-1
-1	-1	1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1	-1

Image is represented as matrix of pixel values... and computers are literal!

We want to be able to classify an X as an X even if it's shifted, shrunk, rotated, deformed.